

Course: Physics-II

B.Tech(CSE) - Term-III

A

Project on

Cardio Care: Real Time Heart Rate Monitoring

Submitted in fulfilment of the award of the

Bachelor of Technology

in

Department of Computer Science and Engineering

by

B. VARSHINI

AUU23EGCSE065

Under the esteemed guidance of

Dr. N. Rakesh Chandra

Associate Professor



**DEPARTMENT OF PHYSICS
SCHOOL OF LIBERAL ARTS AND SCIENCES**

**AURORA HIGHER EDUCATION AND RESEARCH ACEDAMY
(Deemed to be University)**

Parvathapur, Uppal, Hyderabad-500 098

(2023-24)

CERTIFICATE

This is to certify that the project entitled " **Cardio Care: Real Time Heart Rate Monitoring**" has been submitted by **B. VARSHINI** bearing roll number **AUU23EGCSE065** in fulfilment for laboratory project in Physics Laboratory is a record of bonafide work carried out by him/her under my guidance and supervision.

Date: 20-05-2024
Hyderabad

Dr. N. Rakesh Chandra
Associate Professor
Department of Physics
School of Liberal Arts and Sciences

CERTIFICATE

This is to certify that the project entitled " **Cardio Care: Real Time Heart Rate Monitoring**" has been submitted by **B. VARSHINI** bearing roll number **AUU23EGCSE065** in fulfilment for laboratory project in Physics Laboratory is a record of bonafide work carried out by him under the guidance and supervision of **Dr. N. Rakesh Chandra**.

Date: 20-05-2024
Parvathapur, Hyderabad

Dr. N. Rakesh Chandra
Head of the Department
Department of Physics
School of Liberal Arts and Sciences

Acknowledgement

We profoundly grateful to express our deep sense of gratitude and respect towards our guide, **Dr. N. Rakesh Chandra, Associate Professor, Department of Physics, School of Liberal Arts and Sciences, Aurora Deemed-to-be University, Parvathapur**, for his excellent guidance right from selection of project and his valuable suggestions throughout the project work. We are thankful to him for giving us the opportunity to work in the laboratory at any time. His constant encouragement and support have been the cause for us to succeed in completing this project in university. He has given us tremendous support on both the technical and moral front.

We are thankful to **Dr. M.C. Ajay Kumar, and Dr. N. Rakesh Chandra, Department of Physics, School of Liberal Arts and Sciences**, for their valuable suggestions and support in completion of the project.

We are thankful to **Dr. Srilatha Chepure**, Registrar, **AURORA UNIVERSITY** for the support during and till the completion of the project.

We extend our thanks to University Management for their support and encouragement for the success of our project.

INDEX

S. No.	Content	Page No.
1	Introduction	6
2	Project Overview	7-8
3	Demonstration Flow	8-12
4	Live Demonstration	12
5	Key Features	12-14
6	Technical Details	14

1. Introduction:

Monitoring heart rate is a critical aspect of health and fitness tracking. With advancements in sensor technology and microcontroller platforms like Arduino, it has become easier for hobbyists, students, and professionals to create their own heart rate monitoring systems. This project focuses on using a pulse sensor (SEN-11574) and an Arduino to measure heart rate, visualize data, and even use it to control other devices like LEDs.

Purpose and Objectives

The primary purpose of this project is to demonstrate how to use the SEN-11574 pulse sensor with an Arduino to measure and monitor heart rate. By the end of this project, you will be able to:

1. **Understand the Working Principle:** Learn how the pulse sensor detects heart rate through changes in light absorption due to blood flow.
2. **Connect the Pulse Sensor to Arduino:** Follow step-by-step instructions to set up the hardware connections between the pulse sensor and the Arduino.
3. **Program the Arduino:** Use provided example sketches to read data from the pulse sensor and perform various tasks such as blinking an LED with your heartbeat and plotting the heart rate on the Serial Monitor.
4. **Visualize Heart Rate Data:** Implement real-time data visualization using Processing, a flexible software sketchbook and a language for learning how to code within the context of the visual arts.

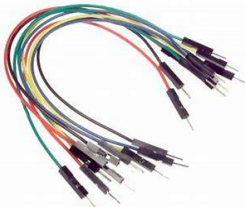
Significance of the Project

Heart rate monitoring is crucial in many fields including healthcare, sports, and fitness. Understanding and tracking heart rate can provide valuable insights into a person's health status and physical condition. For instance:

- **Healthcare:** Regular heart rate monitoring can help detect abnormalities and assist in diagnosing heart-related conditions.
- **Fitness:** Athletes and fitness enthusiasts can use heart rate data to optimize their workouts, ensuring they train within their target heart rate zones for maximum efficiency.
- **Education:** This project serves as an excellent educational tool, providing hands-on experience with bio-sensing technology, signal processing, and data visualization.

Moreover, this project highlights the accessibility and versatility of Arduino, making it a perfect platform for both beginners and advanced users interested in creating interactive and responsive systems. By engaging with this project, you not only gain technical skills but also contribute to the growing trend of DIY health monitoring solutions, promoting a proactive approach to health and wellness.

2. Project Overview:

S.no	Component Image	Name of the Component	Specifications	Purpose
1		Arduino UNO Board	Microcontroller: ATmega328P Operating Voltage: 5V Digital I/O Pins: 14 Analog Input Pins: 6 Flash Memory: 32 KB (0.5 KB used by bootloader) SRAM: 2 KB EEPROM: 1 KB	Main control unit for data processing and communication
2		Pulse sensor	Sensor type: Photoplethysmography (PPG) Operating Voltage: 3.3V - 5V Output: Analog signal proportional to heart rate Compatible with Arduino	Measures heart rate in real-time
4		Processing software	Compatible with various operating systems (Windows, macOS, Linux), Open-source programming language and integrated development environment (IDE).	Enables visualization, animation, and interaction in electronic projects., Used for data processing, simulation, and creating graphical user interfaces (GUIs).
5		Jumper Wires	Length: Various Type: Male to Male Connectors: Dupont connectors Material: Copper wire with insulation	Facilitates electrical connections between components, allowing for a modular and flexible setup

- **Heartbeat Detection and LED Blinking:**
 - The project starts with a simple example where the Arduino reads the pulse sensor data and blinks the onboard LED in sync with the heartbeat. This helps in understanding the basic working of the sensor and provides immediate visual feedback.
- **Serial Plotting:**
 - By utilizing the Serial Plotter in the Arduino IDE, you can visualize the raw heart rate signal data in real-time. This feature allows you to see how the sensor data fluctuates with each heartbeat, helping in fine-tuning the sensor setup and threshold values for accurate detection.
- **BPM Calculation and Display:**
 - The project includes a sketch to calculate the BPM (beats per minute) from the pulse sensor data. The BPM value is printed on the Serial Monitor, providing a quantitative measure of the heart rate.
- **Processing Visualization:**
 - For more advanced visualization, the project uses Processing to create a graphical interface that displays the heart rate signal, BPM, and IBI in real-time. This interactive visualization helps in better understanding the heart rate pattern and the performance of the sensor.
- **Real-time Data Monitoring:**
 - With the Processing sketch, you can monitor heart rate data in real-time, offering an engaging way to see the immediate impact of different activities or conditions on the heart rate.
- **Interactive Learning and Troubleshooting:**
 - The project is designed with multiple examples sketches to provide a comprehensive learning experience. From basic heartbeat detection to advanced data visualization, each example helps in building a deeper understanding of bio-sensing and signal processing.
 - Additionally, the example sketches are structured to help in troubleshooting and improving the accuracy of the sensor readings.

3. Demonstration Flow:

Setting Up the Hardware

- **Component Display:** Showing all the components used in the project.
- **Circuit Assembly:** Demonstrating how to connect the pulse sensor to the Arduino using a breadboard and connecting wires. Ensure the connections are clear:
 - Pulse sensor VCC to Arduino 5V
 - Pulse sensor GND to Arduino GND
 - Pulse sensor Signal to Arduino Analog Pin A0
- **LED Connection:** Connecting an LED to an appropriate digital pin (e.g., pin 13) for heartbeat indication.



Initial Testing with Arduino IDE

- **Arduino Setup:** Opening the Arduino IDE and explain its basic interface.
- **Example Sketch:** Load a simple example sketch to read the pulse sensor data and blink the onboard LED.

Code Walkthrough:

```
sketch_may20a | Arduino IDE 2.3.2
File Edit Sketch Tools Help

sketch_may20a.ino
1 // Variables
2 int PulseSensorPurplePin = 0; // Pulse Sensor PURPLE WIRE connected to ANALOG PIN 0
3 int LED13 = 13; // The on-board Arduino LED
4
5
6 int Signal; // holds the incoming raw data. Signal value can range from 0-1024
7 int Threshold = 550; // Determine which Signal to "count as a beat", and which to ignore.
8
9
10 // The SetUp Function:
11 void setup() {
12   pinMode(LED13,OUTPUT); // pin that will blink to your heartbeat!
13   Serial.begin(9600); // Set's up Serial Communication at certain speed.
14 }
15
16
17 // The Main Loop Function
18 void loop() {
19
20   Signal = analogRead(PulseSensorPurplePin); // Read the PulseSensor's value.
21   // Assign this value to the "Signal" variable.
22
23   Serial.println(Signal); // Send the Signal value to Serial Plotter.
24
25
26   if(Signal > Threshold){ // If the signal is above "550", then "turn-on" Arduino's on-Board LED.
27     digitalWrite(LED13,HIGH);
28   } else {
29     digitalWrite(LED13,LOW); // Else, the signal must be below "550", so "turn-off" this LED.
30   }
31
32   delay(10);
33 }
```

Firstly, some variables are defined that include the GPIO through which the sensor's signal pin is connected with. It is A0. Next, the built-in LED's GPIO pin number is stated. It is GPIO13. The signal variable of data type integer holds the ADC data and the threshold variable will help in differentiating a valid heartbeat. By default, it is set to 550.

```
int PulseSensorPurplePin = 0;  
int LED13 = 13;  
int Signal;  
int Threshold = 550;
```

setup()

Inside the setup() function, the serial connection is opened at a baud rate of 9600.

```
Serial.begin(9600);
```

Using the pinMode() function, the built-in LED will be configured as an output pin.

```
pinMode(LED13,OUTPUT);
```

loop()

Inside the loop() function, first by using analogRead() on the A0 pin the ADC signal data will get saved in the 'Signal' variable. This is the same variable that we defined earlier.

```
Signal = analogRead(PulseSensorPurplePin);
```

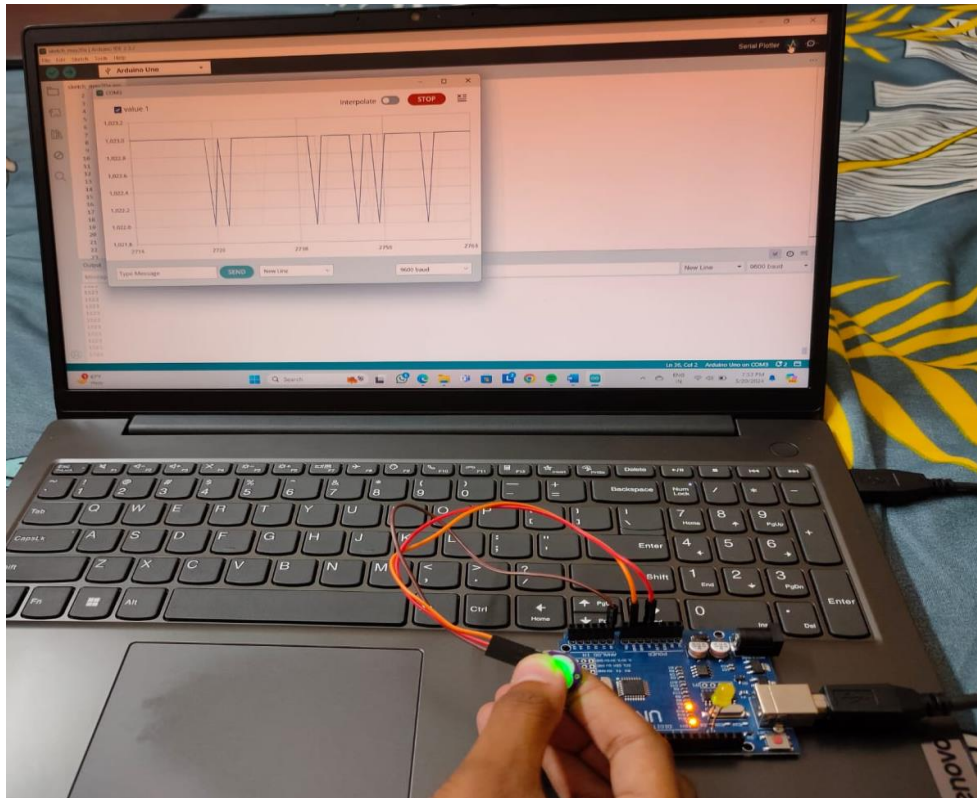
Next, print this value in the serial monitor/serial plotter for debugging purposes and can also be used to set the threshold value.

```
Serial.println(Signal)
```

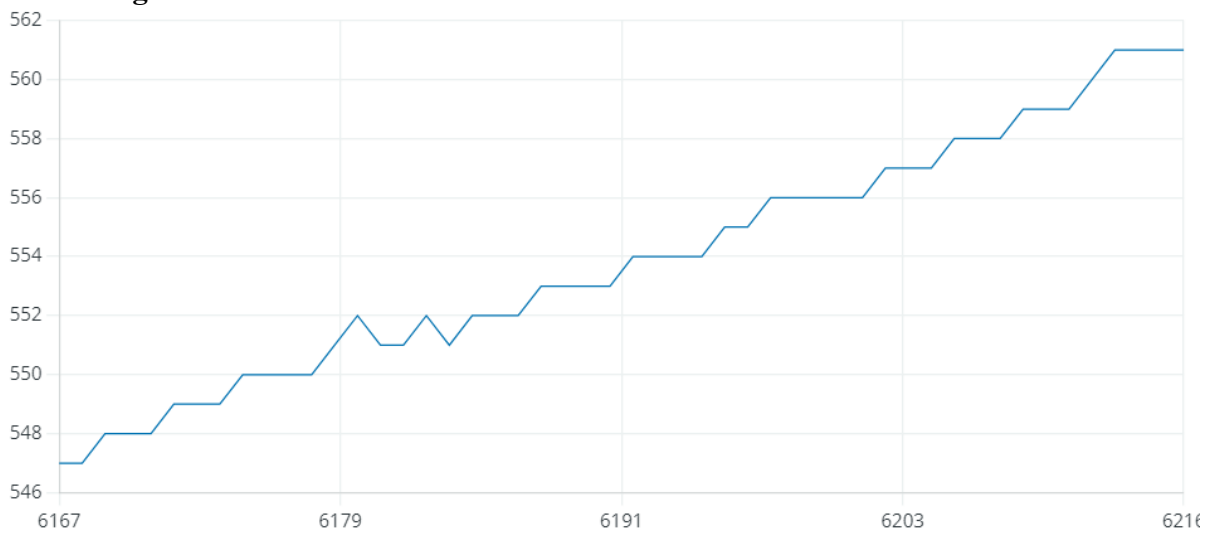
The following lines of code will check whether the value saved in the signal variable is greater than the set threshold value. If it is, then turn the built-in LED ON. Otherwise, turn it OFF.

```
if(Signal > Threshold){  
    digitalWrite(LED13,HIGH);  
} else {  
    digitalWrite(LED13,LOW);  
}  
delay(10);
```

Upload and Run:



Visualizing Data with Serial Plotter



Calculating and Displaying BPM



5. Key Features:

1. Understanding the Working Principle: The pulse sensor detects heart rate through changes in light absorption due to blood flow.

Explanation:

- **Photoplethysmography (PPG):** The pulse sensor operates on the principle of photoplethysmography. It uses a light source (usually an LED) and a photodetector to measure the amount of light that is either absorbed or reflected by the blood flowing through the skin.
- **Heartbeats Detection:** Each heartbeat causes a surge of blood in the capillaries, leading to variations in light absorption. The sensor detects these variations as changes in the intensity of the reflected light.
- **Data Output:** These changes are converted into an analog signal that can be read by the Arduino, which then processes the data to calculate the heart rate.

2. Connecting the Pulse Sensor to Arduino: Following step-by-step instructions to set up the hardware connections between the pulse sensor and the Arduino.

Step-by-Step Instructions:

- **Gather Components:** Ensure you have the pulse sensor, Arduino board, breadboard, connecting wires, and an LED.
- **Connection Guide:**
 - **Pulse Sensor to Arduino:**
 - VCC (Red Wire) to Arduino 5V
 - GND (Black Wire) to Arduino GND
 - Signal (Purple Wire) to Arduino Analog Pin A0
 - **LED to Arduino** (for visual heartbeat indication):
 - Connect the long leg (anode) of the LED to a current-limiting resistor (220 ohms) and then to Arduino Digital Pin 13.
 - Connect the short leg (cathode) of the LED to Arduino GND.

3. Programming the Arduino: Using provided example sketches to read data from the pulse sensor and perform various tasks.

Example Sketches:

- **Basic Data Reading:**
 - **Sketch:** Loading a simple sketch to read the analog values from the pulse sensor.
 - **Code Overview:** The sketch reads the analog input from the sensor pin and prints the values to the Serial Monitor.
 - **Upload and Run:** Upload the sketch to the Arduino and open the Serial Monitor to see the raw data values.
- **Heartbeat LED Blink:**
 - **Sketch:** A more advanced sketch to blink an LED in sync with the heartbeat.
 - **Code Walkthrough:** The code includes a threshold value to detect peaks in the analog signal, representing heartbeats, and triggers the LED to blink.
 - **Upload and Test:** Upload the sketch and observe the LED blinking in sync with your heartbeat.
- **BPM Calculation:**
 - **Sketch:** An advanced sketch that calculates beats per minute (BPM).
 - **Code Explanation:** The code processes the analog input to detect heartbeats and calculates the time interval between beats to determine BPM.
 - **Real-Time Display:** Upload the sketch and monitor the BPM value in the Serial Monitor.

4. Visualizing Heart Rate Data: Implementing real-time data visualization using Processing, a flexible software sketchbook and language for learning how to code within the context of the visual arts.

Processing Setup:

- **Download and Install Processing:** Install Processing from its official website.
- **Create a New Sketch:** Open Processing and create a new sketch.

Processing Sketch:

- **Code:** Write or load a pre-written Processing sketch designed to visualize heart rate data.
- **Data Communication:** The Arduino sends serial data to Processing.

- **Graphical Display:** Processing receives the data and visualizes it, showing real-time heart rate signal, BPM, and IBI (Inter-Beat Interval).
- **Running the Sketch:**
 - **Connect Arduino:** Ensure the Arduino is connected and running the appropriate sketch to send data.
 - **Run Processing Sketch:** Run the Processing sketch and observe the graphical representation of the heart rate data.
 - **Interactive Visualization:** The visualization should update in real-time, reflecting changes in heart rate and allowing for a clear understanding of the data.

6. Technical Details:

Technology Stack Overview:

- **SEN-11574 Pulse Sensor:** This photoplethysmography (PPG) sensor detects heart rate by measuring changes in light absorption as blood pulses through the capillaries. The sensor outputs an analog signal that corresponds to the heartbeat.
- **Arduino Microcontroller:** The Arduino serves as the central processing unit, reading the analog signals from the pulse sensor and converting them into digital data. It uses the AnalogRead function to capture the sensor data and processes it to detect heartbeats and calculate the beats per minute (BPM).
- **Processing Software:** Processing is used to create a real-time graphical representation of the heart rate data. This software communicates with the Arduino via serial communication, receiving data and visualizing it in a user-friendly format.

Technical Complexities and Solutions:

- **Signal Noise and Filtering:** One of the significant challenges is dealing with noise in the analog signal from the pulse sensor, which can lead to inaccurate heartbeat detection. This was addressed by implementing software-based filtering techniques within the Arduino code. Algorithms such as smoothing and threshold detection were used to identify valid heartbeats amidst noisy data.
- **Serial Communication:** Ensuring reliable data transmission between the Arduino and Processing posed another challenge. This was overcome by optimizing the serial communication protocol, ensuring that data packets were correctly formatted and transmitted without loss or corruption.
- **Real-Time Data Processing:** Handling real-time data processing and visualization required efficient coding practices. In Processing, optimized loops and graphical functions were used to ensure that the heart rate data was displayed without lag. The Arduino sketch was designed to process incoming data swiftly and accurately, ensuring minimal delay in heartbeat detection and BPM calculation.
- **User Interaction and Display:** Creating an intuitive and informative user interface in Processing involved designing graphical elements that could dynamically update based on real-time data. This included visual indicators of heartbeat detection, graphical plots of the heart rate over time, and numeric displays of BPM.